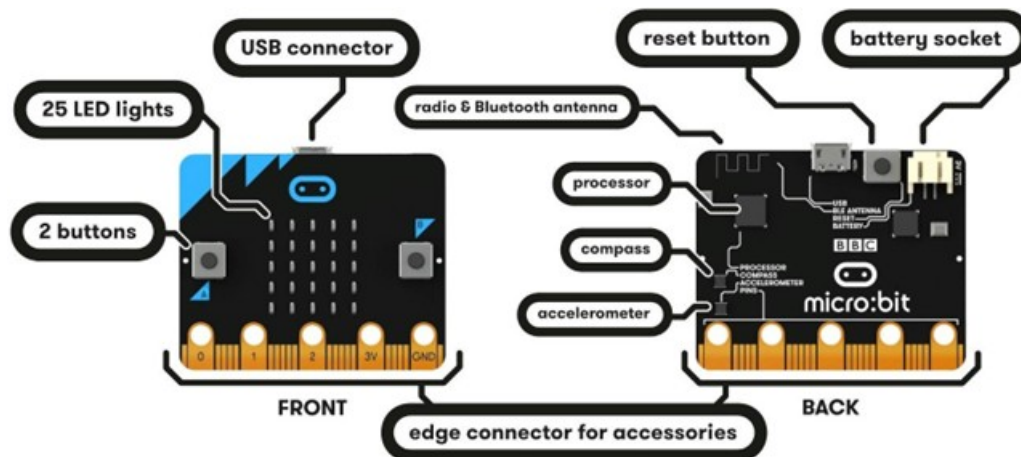


# Conocer el Hardware

Empecemos por el principio, el hardware. A continuación podemos ver una representación de la placa con la que vamos a trabajar en este manual.



La micro:bit tiene las siguientes características hardware:

- 25 LED programables individuales
- 2 botones programables
- Pines de conexión física
- Sensores de luz y temperatura
- Sensores de movimiento (acelerómetro y brújula)
- Comunicación inalámbrica, vía radio y Bluetooth
- Interfaz USB

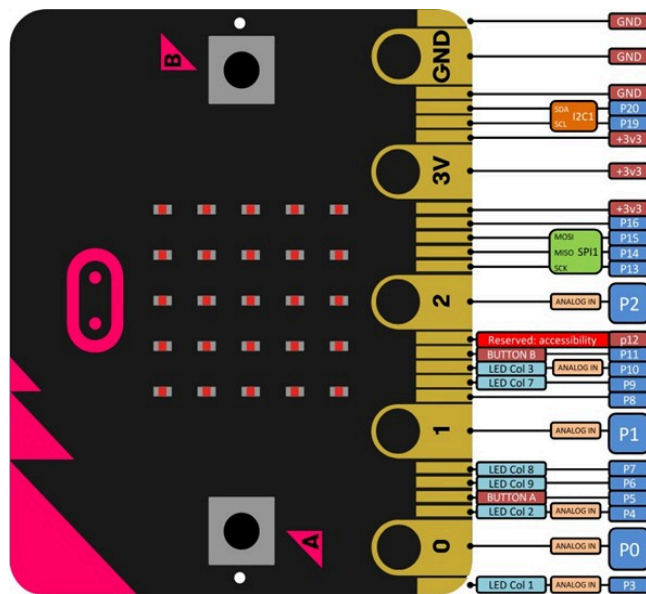
Para más información sobre la micro:bit, podéis visitar la página:

<https://microbit.org/guide/features/>.

No es necesario que los principiantes dominen esta sección, pero sí es necesario tener una idea general. Sin embargo, si queremos ser desarrolladores, la información sobre el hardware resultará muy útil. Encontraremos información detallada sobre el hardware de la micro:bit [aquí](#).

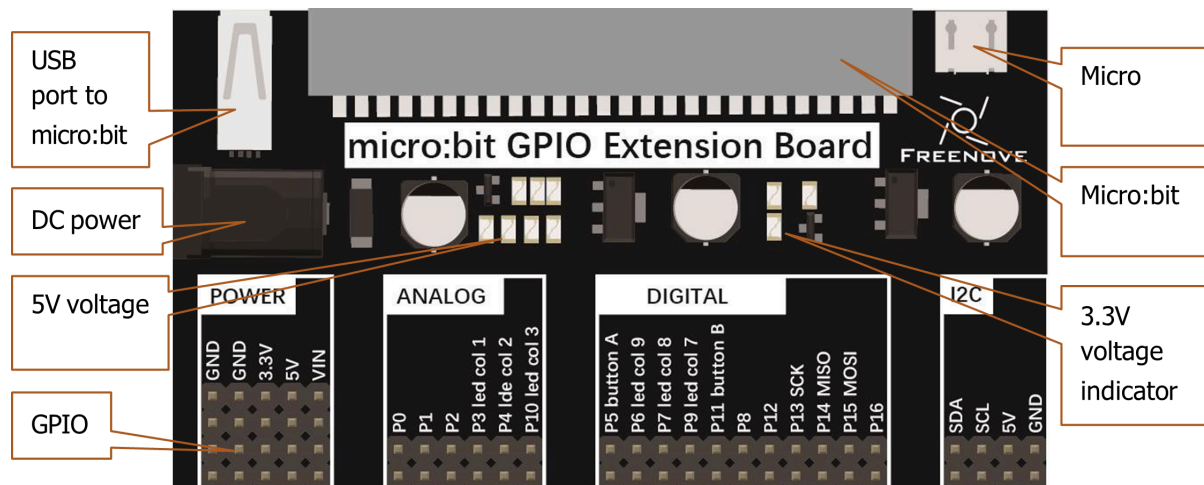
## GPIO

La GPIO, es decir, los pines de entrada/salida de uso general, son un componente importante de la micro:bit para conectar dispositivos externos. Todos los sensores y dispositivos de la placa se comunican entre sí a través de los GPIO del micro:bit. A continuación se muestra el diagrama de numeración y funciones:



## GPIO - Placa de extensión

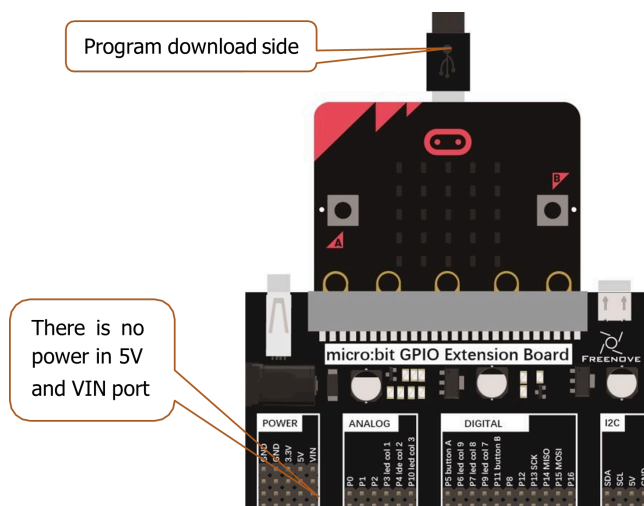
La placa de extensión es un accesorio que permite conectar la micro:bit a otros dispositivos electrónicos conectado a la placa micro:bit a través del socket correspondiente. Esta placa proporciona una forma más fácil de acceder a los pines GPIO de la micro:bit, lo que facilita la conexión de sensores, actuadores y otros componentes electrónicos. A continuación se muestra un ejemplo de una placa de extensión:



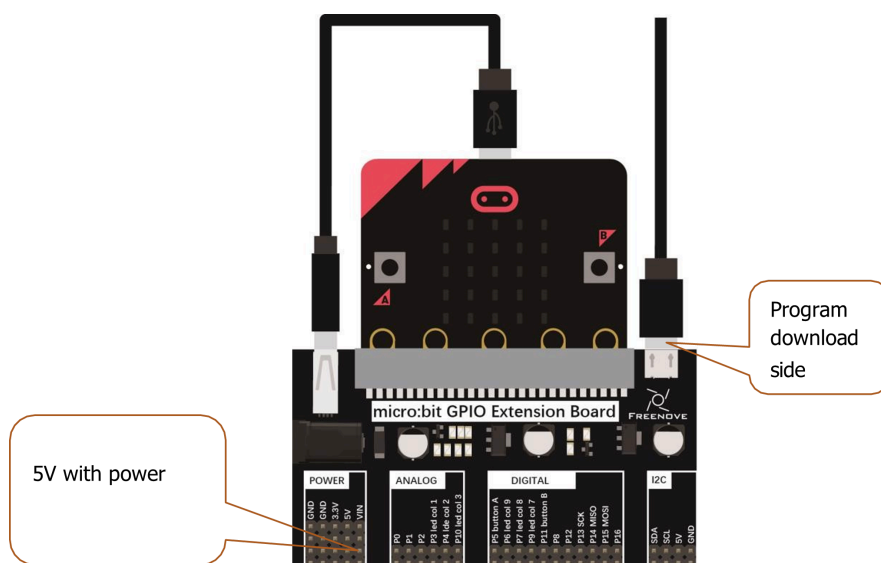
En esta extensión se han añadido puertos de entrada - salida adicionales con voltajes de 5V y VIN(9V) para suplir los requerimientos de algunos dispositivos.

## Cómo usar la extensión

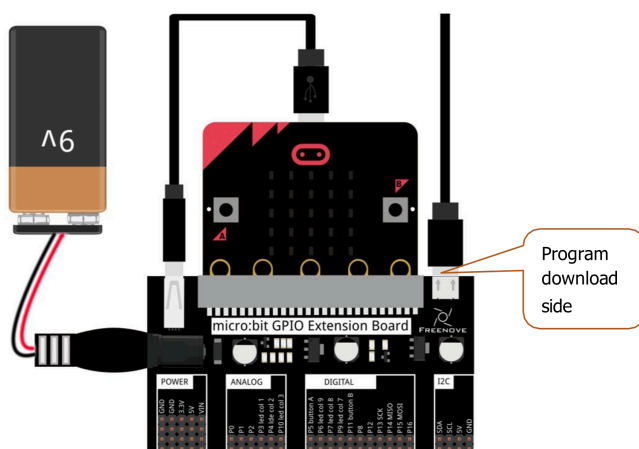
Si no es necesario utilizar los voltajes extendidos podemos hacer uso de la placa de la siguiente manera:



Si los periféricos necesitan 5V, pero no mucha potencia, la siguiente imagen muestra cómo hacerlo:



Por último, si las necesidades son elevadas, el diagrama siguiente muestra la configuración adecuada:



# Programación de la Micro::bit 2

Con este manual vamos a aprender a programar la Micro::bit 2, un microcontrolador diseñado para la educación y la experimentación en electrónica y programación. La Micro::bit 2 cuenta con una variedad de sensores, botones, una pantalla LED y conectividad Bluetooth, lo que la hace ideal para proyectos interactivos. Utilizaremos como base el lenguaje Rust, que es un lenguaje de programación moderno y seguro, para crear programas que interactúen con los componentes de la Micro::bit 2.

## Referencias

Antes de nada unas pocas referencias:

- [Documentación oficial de la Micro::bit 2](#)
- [Documentación de Rust](#)
- [micro::bit v2 Embedded Discovery Book \(castellano\)](#)
- [Rust Embedded Book](#)

## Instalación del entorno de desarrollo

Para programar la Micro::bit 2 con Rust, necesitamos configurar nuestro entorno de desarrollo. Antes de pasar a la instalación real, enumeraremos los requisitos que he probado en este manual:

- Rust 1.79.0 o una toolchain más reciente.
- `gdb-multiarch`. Herramienta de depuración. La versión más antigua que hemos probado es la 10.2, pero es probable que otras versiones también funcionen. Si la distribución/plataforma no tiene `gdb-multiarch` disponible, podemos usar `arm-none-eabi-gdb` para depurar. Además, algunos binarios de `gdb` están contruidos con capacidades multiplataforma: se puede encontrar más información sobre esto en el capítulo de depuración de este libro.
- `[ cargo-binutils ]`. Versión 0.3.6 o posterior. `[ cargo-binutils ]`: <https://github.com/rust-embedded/cargo-binutils>
- `[ probe-rs-tools ]`. Versión 0.24.0 o más reciente. `[ probe-rs-tools ]`: <https://probe.rs/docs/overview/about-probe-rs/>
- `minicom` en Linux y macOS. Se ha probado la versión: 2.7.1. Esperamos que versiones posteriores también funcionen.
- `PuTTY` en Windows.

## Instalación de Rust

Para instalar Rust, podemos utilizar `rustup`, que es la herramienta oficial para gestionar las versiones de Rust. Si no tenemos `rustup` instalado, seguiremos las instrucciones en la [página oficial de Rust](#).

---

**Instrucciones de la página:** Para empezar a usar Rust, descarga el instalador, ejecuta el programa y sigue las instrucciones que aparecen en pantalla. Es posible que tengas que instalar las herramientas de compilación de Visual Studio C++ cuando se solicite. Si no utilizas Windows, consulta "Otros métodos de instalación".

---

```
$ rustc -V
rustc 1.94.0 (4a4ef493e 2026-03-02)
```

Una vez instalado `rustup`, tenemos que instalar la toolchain de Rust con el comando:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará automáticamente la cadena de compilación si se lo pedimos. Explicaremos más adelante por qué es necesario instalar esta toolchain específica.

## Instalación bajo Windows

La mayoría de ordenadores educativos utilizan Windows, por lo que vamos a detallar los pasos para instalar Rust en este sistema operativo. Si estamos usando otra plataforma recomiendo leer el libro de micro::bit v2 Embedded Discovery Book.

### **`gdb-multiarch (arm-none-eabi-gdb)`**

---

Hemos descargado e instalado el siguiente archivo de la página oficial de Arm: [Aquí](#)

---

Arm proporciona ejecutables ( `.exe` ) para Windows. Se pueden encontrar en [gcc](#), solo hay que seguir las instrucciones. Justo antes de que finalice el proceso de instalación, hay que marcar/seleccionar la opción “Añadir ruta a la variable de entorno”. Si no aparece dicha opción se tendrá que hacer desde el diálogo del sistema y añadir la siguiente ruta para la instalación por defecto:

---

```
C:\Program Files\Arm\GNU Toolchain mingw-w64-x86_64-arm-none-eabi\bin
```

---

A continuación, comprobaremos que las herramientas se encuentran en el `%PATH%`:

```
$ arm-none-eabi-gcc -v
(..)
gcc version 15.2.1 20251203 (Arm GNU Toolchain 15.2.Rel1 (Build arm-15.86))
```

### **`cargo-binutils`**

```
$ rustup component add llvm-tools
$ cargo install cargo-binutils --vers '^0.4'
$ cargo size --version
cargo-size 0.4.0
```

### **`probe-rs-tools`**

**NOTA** Si existen en el sistema versiones anteriores de `probe-run`, `probe-rs` o `cargo-embed` instaladas, hay que eliminarlas antes de seguir adelante, ya que podrían causar problemas. En particular, `probe-run` ya no existe oficialmente. Hay que eliminarlas si es necesario:

```
$ cargo uninstall cargo-embed
$ cargo uninstall probe-run
$ cargo uninstall probe-rs
$ cargo uninstall probe-rs-cli
```

Para instalar `probe-rs-tools`, hay que seguir las instrucciones de la página oficial en <https://probe.rs>. En mi caso la instalación mediante cargo no me ha dado ningún tipo de problema.

```
$ cargo install probe-rs-tools
```

```
Finished `release` profile [optimized] target(s) in 3m 34s
Installing C:\Users\arcipreste\.cargo\bin\cargo-embed.exe
Installing C:\Users\arcipreste\.cargo\bin\cargo-flash.exe
Installing C:\Users\arcipreste\.cargo\bin\probe-rs.exe
Installed package `probe-rs-tools v0.31.0` (executables `cargo-embed.exe`, `cargo-
flash.exe`, `probe-rs.exe`)
```

```
C:\Users\...>probe-rs
The probe-rs CLI
```

Instalar la herramienta `probe-rs-tools` configurará en nuestro ordenador diversas herramientas muy útiles, incluyendo `probe-rs` y `cargo-embed` (que normalmente se ejecutan como un comando de Cargo). Debemos comprobar que todo funciona correctamente antes de pasar a la siguiente sección.

```
$ cargo embed --version
cargo embed 0.31.0 (git commit: crates.io)
```

## PuTTY

Se puede descargar `putty.exe` desde [aquí](#) y añadirlo al `%PATH%`.

## Primer ejemplo: “Hola, mundo”

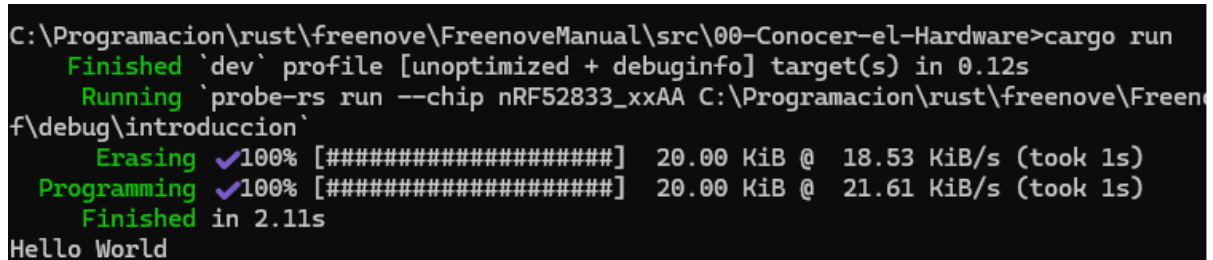
Vamos a crear nuestro primer programa para la Micro::bit 2, que mostrará el mensaje “Hola, mundo” en el terminal. Para ello, seguiremos los siguientes pasos:

- Conectamos la placa a un puerto USB de nuestro ordenador. Comprobamos que se abre una ventana con el contenido de la MB2.
- Abrimos un terminal y nos situamos en el directorio del capítulo inicial:

```
$ cd src/00-Conocer-el-Hardware/
```

- Ejecutamos el siguiente comando para compilar y ejecutar el programa:

```
$ cargo run
```



```
C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware>cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `probe-rs run --chip nRF52833_xxAA C:\Programacion\rust\freenove\FreenoveManual\src\00-Conocer-el-Hardware\src\main.rs`
Erasing ✓100% [#####] 20.00 KiB @ 18.53 KiB/s (took 1s)
Programming ✓100% [#####] 20.00 KiB @ 21.61 KiB/s (took 1s)
Finished in 2.11s
Hello World
```

El código fuente del programa se encuentra en el archivo `src/main.rs`. A continuación, se muestra el contenido del archivo:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

# Un poco más de programación

## Comprendiendo el primer programa

Vamos a echar un vistazo a nuestro primer programa. Comprobemos el fichero `src/main.rs`:

```
#![no_main]
#![no_std]

use cortex_m::asm::{nop};
use nrf52833_pac as _;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

use cortex_m_rt::entry;

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Hello World");
    loop {
        nop();
    }
}
```

Los programas para microcontroladores son diferentes de los programas estándar en dos aspectos: `#![no_std]` y `#![no_main]`.

El atributo `no_std` indica que este programa no usará la biblioteca estándar de Rust, la cual asume un sistema operativo subyacente; el programa utilizará en su lugar el crate `core`, un subconjunto de `std` que puede ejecutarse en sistemas hardware directamente.

El atributo `no_main` indica que este programa no usará la interfaz estándar 'main', que está diseñada para aplicaciones de línea de comandos que reciben argumentos. En lugar del 'main' estándar, usaremos el atributo 'entry' del crate `cortex-m-rt` para definir un punto de entrada personalizado. En este programa hemos definido el punto de entrada como `main`, pero se podría haber usado cualquier otro nombre. La función del punto de entrada debe tener la firma `fn() -> !`; este tipo indica que la función no termina. Significa que el programa nunca finaliza: si el compilador detecta que esto sería posible, se negará a compilarlo.

Si observamos con cuidado el directorio del proyecto notaremos que hay un directorio `.cargo` posiblemente oculto en el proyecto. Este directorio contiene un archivo de configuración de Cargo `.cargo/config.toml`.

```
[build]
target = "thumbv7em-none-eabihf"

[target.thumbv7em-none-eabihf]
runner = "probe-rs run --chip nRF52833-xxAA"
rustflags = [
    "-C", "linker=rust-lld",
]
```

Este fichero modifica el proceso de enlace para adaptar la disposición de memoria del programa a los requisitos del dispositivo de desarrollo. Este proceso de enlace es un requisito del crate `cortex-m-rt`. El archivo `.cargo/config.toml` también le dice a Cargo cómo construir y ejecutar el código en nuestra MB2.

Hay también un fichero `Embed.toml`:



```
[default.probe]
protocol = "Swd"

[default.general]
chip = "nrf52833_xxAA"

[default.reset]
halt_afterwards = true

[default.rtt]
enabled = true

[default.gdb]
enabled = false
```

Este fichero informa a `cargo-embed` que:

- Trabaja con un chip NRF52833.
- Queremos detener la ejecución en el chip después de flashearlos, por lo que nuestro programa se detiene antes de `main`.
- Queremos deshabilitar/habilitar RTT. RTT es un protocolo que permite al chip enviar texto a un depurador. Ya has visto RTT en acción fue la primera versión del programa inicial.
- Queremos Deshabilitar/habilitar GDB. Este paso es necesario para procesos de depuración tal y como veremos al final de este capítulo.

## Generando el binario

El primer paso es construir el binario. Dado que el microcontrolador tiene una arquitectura diferente a la de nuestro ordenador, tendremos que realizar una compilación cruzada. Hacerlo en Rust es tan simple como pasar un flag extra `--target` a `rustc` o Cargo. La parte complicada es averiguar el argumento de ese flag: el *nombre* del destino.

Como ya hemos visto, el microcontrolador del micro:bit tiene un procesador Cortex-M4F. `rustc` sabe cómo compilar de forma cruzada para la arquitectura Cortex-M y proporciona varios destinos que cubren las diferentes familias de procesadores dentro de esa arquitectura:

- `thumbv6m-none-eabi`, para los procesadores Cortex-M0 y Cortex-M1
- `thumbv7m-none-eabi`, para el procesador Cortex-M3
- `thumbv7em-none-eabi`, para los procesadores Cortex-M4 y Cortex-M7
- `thumbv7em-none-eabihf`, para los procesadores Cortex-M4F y Cortex-M7F
- `thumbv8m.main-none-eabi`, para los procesadores Cortex-M33 y Cortex-M35P
- `thumbv8m.main-none-eabihf`, para los procesadores Cortex-M33F y Cortex-M35PF

“Thumb” aquí se refiere a una versión del conjunto de instrucciones Arm que tiene instrucciones más pequeñas para reducir el tamaño del código. La denominación `hf / F` significa que implementa aceleración de punto flotante por hardware.

Para la MB2, micro:bit v2, queremos el destino `thumbv7em-none-eabihf`.

Antes de realizar la compilación cruzada, es necesario descargar una versión precompilada de la biblioteca estándar (en realidad, una versión reducida de ella) en el host local. Se hace usando `rustup`:

```
$ rustup target add thumbv7em-none-eabihf
```

Solo hay que hacerlo una vez; `rustup` actualizará este destino (reinstalando un nuevo componente de la biblioteca estándar `rust-std` que contiene la biblioteca `core` que usamos) cada vez que actualicemos la cadena de compilación. Por lo tanto, **se puede omitir este paso si ya se agregó la `toolchain` necesaria anteriormente.**

Con el componente `rust-std` en su lugar, ya es posible compilar el programa de forma cruzada usando Cargo. Nos aseguraremos de estar en el directorio `src/00-Conocer-el-Sistema`, luego lo construimos. Este código inicial es un ejemplo, así que lo compilamos como tal.

```
$ cargo build
  Compiling semver-parser v0.7.0
  Compiling proc-macro2 v1.0.86
  ...

Finished dev [unoptimized + debuginfo] target(s) in 33.67s
```

Ok, ya tenemos un ejecutable. ¡No hará mucho!, imprime el mensaje “Hello, World!”.

## Flashearlo

Flashear es el proceso de mover el programa a la memoria del microcontrolador. Una vez hecho, el microcontrolador ejecutará el programa cada vez que se encienda.

El programa será el único existente en la memoria. Por esto me refiero a que no hay nada más ejecutándose en el microcontrolador: ni un sistema operativo, ni un “daemon”, nada. Nuestro programa tiene control total sobre el dispositivo.

Pasarlo al microcontrolador es muy simple, gracias a `cargo embed\run`.

```
$ cargo run
(...)
Erasing sectors ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
4.21KiB/s (eta 0s )
Programming pages ✓ [00:00:00]
[#####] 2.00KiB/ 2.00KiB @
2.71KiB/s (eta 0s )
Finished flashing in 0.608s
```

Es importante fijarse que si queremos depurar se tiene que lanzar con `cargo embed`, y de esta forma no termina después de mostrar la última línea. Esto es intencionado: no hay que cerrar `cargo embed`, ya que lo necesitamos en este estado para depurar.

## Depuración de programas

La depuración de programas es una parte fundamental del desarrollo de software, especialmente cuando se trabaja con microcontroladores como la Micro::bit. Para un funcionamiento correcto de la depuración, necesitamos configurar el proyecto para que utilice `gdb` en lugar de `rtt`. Para ello, editamos el archivo `Embed.toml` y establecemos `enabled = false` para `rtt` y `enabled = true` para `gdb`. El contenido del archivo debe ser el siguiente:

```
[default.rtt]
enabled = false

[default.gdb]
enabled = true
```

Tras modificar el fichero `Embed.toml` en el directorio `00-Conocer-el-Sistema`, ejecutamos pero lanzando la depuración con `cargo embed`:

```
$ cargo embed
```

Nos conectamos a la sesión de depuración con `gdb` desde otra consola (recordar que solo se muestran ejemplos para el SSOO Windows). Para ello, abrimos otra terminal y nos situamos en la raíz del proyecto. Ejecutamos `arm-none-eabi-gdb` con el binario que queremos depurar como argumento:

```
$ cd raiz_del_libro
$ arm-none-eabi-gdb ./target/thumbv7em-none-eabihf/debug/introduccion
```

Si nos da un fallo donde no encuentra el binario, recordar que el directorio `target` se encuentra en la raíz del proyecto, no en `src/00-Conocer-el-Sistema`. Ya dentro de `gdb` nos unimos a la sesión de depuración remota con el comando `target remote` y el puerto que vemos al ejecutar con `cargo embed` (en este caso, el puerto 1337):

```
(gdb) target remote :1337
Remote debugging using :1337
```

Los puntos de ruptura se pueden usar para detener el flujo normal de un programa. El comando `continue` permitirá que el binario se ejecute libremente *hasta* que alcance un punto de ruptura. En este caso, hasta que alcance la función `main` porque hemos establecido uno allí.

```
(gdb) break main
...
(gdb) continue
...
```

Para ver el contenido de las variables, podemos usar el comando `print` seguido del nombre de la variable. Por ejemplo, para ver el contenido de la variable `x`:

```
(gdb) print x
$1 = 0
```

Si queremos ejecutar el programa paso a paso, podemos usar el comando `next` para avanzar una línea de código a la vez. Esto nos permite observar cómo cambian las variables y el flujo del programa en tiempo real.

```
(gdb) next
16          loop {}
```

El comando `monitor reset` resetea el microcontrolador y lo para en el punto de entrada.

```
(gdb) monitor reset
```

Ya hemos terminado la sesión de depuración. Podemos salir con el comando `quit`.

```
(gdb) quit
A debugging session is active.

        Inferior 1 [Remote target] will be detached.
```

```
Quit anyway? (y or n) y
Detaching from program: $PWD/target/thumbv7em-none-eabihf/debug/meet-your-software,
Remote target
Ending remote debugging.
[Inferior 1 (Remote target) detached]
```

# Resumen

En este capítulo hemos conocido la máquina y como programarla. Hemos aprendido a configurar el SSOO para ejecutar Rust y en concreto programar sistemas embebidos, hemos configurado la toolchain adecuada para la MB2 de Rust y hemos aprendido a compilar, flashear y ejecutar. Además, hemos visto cómo utilizar el depurador para analizar el comportamiento de nuestros programas. En resumen, hemos adquirido los conocimientos necesarios para desarrollar software en sistemas embebidos utilizando Rust.

A partir de ahora, avanzaremos hacia la programación de sistemas embebidos más complejos, a través de proyectos específicos. En concreto cada capítulo se dedicará a una parte del hardware específica.

## Para terminar

Hay un tema que no hemos tratado, el uso de IDEs específicos. Si bien Rust no requiere un IDE para programar, existen algunos que pueden facilitar el desarrollo, como Visual Studio Code o RustRover. Cualquiera de los dos puede ser utilizado para programar en Rust, y ambos ofrecen características como ia, autocompletado, depuración y gestión de proyectos que pueden mejorar la productividad. Sin embargo, es importante recordar que el conocimiento de la línea de comandos y las herramientas básicas sigue siendo fundamental para un desarrollador de sistemas embebidos.

## Como configurar IntelliJ o RustRover

Al editar la configuración de compilación de RustRover, estos son algunos valores no predeterminados:

- Hay que modificar el comando. Cuando se indique que hay que ejecutar `cargo embed FLAGS`, se deberá cambiar el valor predeterminado `run` por el comando `embed FLAGS` si queremos depurar, si solo lo lanzamos no es necesario.
- Se tiene que activar la opción “Emular terminal en la consola de salida”. De lo contrario, el programa no podrá mostrar texto en un terminal.
- Nos aseguraremos que el directorio de trabajo sea `./src/N-nombre_capitulo`, siendo `N-nombre_capitulo` el directorio del capítulo que estamos leyendo. No se puede ejecutar desde el directorio `src 0 FreenoveManual`.

# Capítulo 1 - Matriz de LED

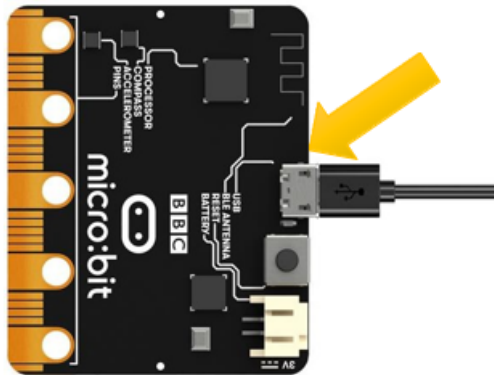
## Descripción del proyecto 1.1

Este proyecto tiene como objetivo crear una animación en la matriz de LEDs de un corazon parpadeando.

### Hardware necesario

- Micro:bit
- Micro Usb

### Esquema de conexión



## Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);
    let light_heart_big = [
        [0, 1, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [0, 1, 1, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let light_heart_small = [
        [0, 1, 0, 1, 0],
        [1, 0, 1, 0, 1],
        [1, 0, 0, 0, 1],
        [0, 1, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        display.show(&mut timer, light_heart_big, 1000);
        timer.delay_ms(1000_u32);
        display.show(&mut timer, light_heart_small, 1000);
        timer.delay_ms(1000_u32);
        // display.clear();
    }
}
```

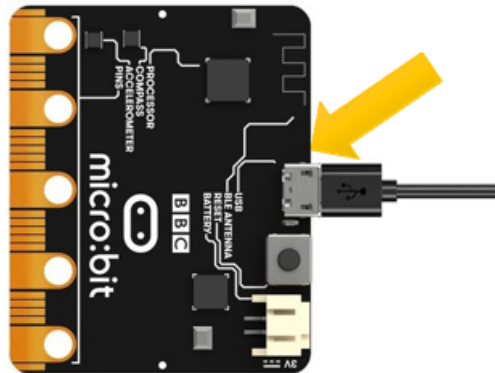
## Descripción del proyecto 1.2

Se pretende desplazar el corazón del segundo frame del ejercicio anterior fuera de la matriz de LEDs.

### Hardware necesario

- Micro:bit
- Micro Usb

## Esquema de conexión



## Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);
    let light_heart_big = [
        [0, 1, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [0, 1, 1, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let mut light_heart_small = [
        [0, 1, 0, 1, 0],
        [1, 0, 1, 0, 1],
        [1, 0, 0, 0, 1],
        [0, 1, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        display.show(&mut timer, light_heart_big, 1000);
        timer.delay_ms(1000_u32);
        for _ in 0..=5 {
            display.show(&mut timer, light_heart_small, 500);
            timer.delay_ms(10_u32);
            lib_cap::rotate_column_matrix(&mut light_heart_small);
        }
    }
}
```

## Comando de ejecución

```
$cargo run --example heart_move
```

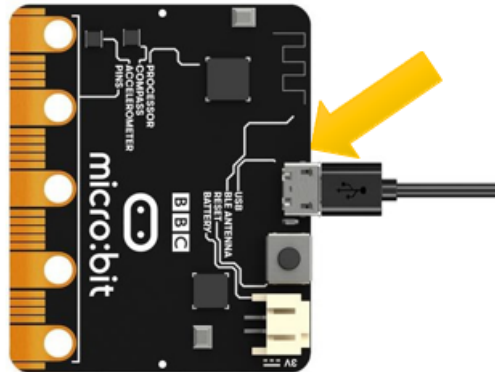
## Descripción del proyecto 1.3

Se pretende mostrar una secuencia de números en la matriz de LEDs, donde cada número se mostrará durante un segundo antes de pasar al siguiente.

### Hardware necesario

- Micro:bit
- Micro Usb

### Esquema de conexión



### Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);

    let mut light_heart_small = [[0; 5];5];
    let data = [1,2,0,5,1];

    loop {
        for num in data {
            lib_cap::number_to_display(num, &mut light_heart_small);
            display.show(&mut timer, light_heart_small, 1000);
            timer.delay_ms(500_u32);
        }
    }
}
```



## Comando de ejecución

```
$cargo run --example numbers
```

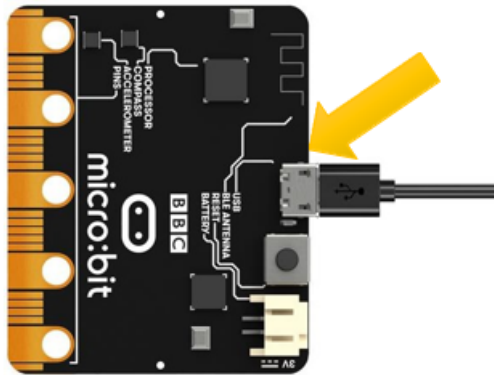
## Descripción del proyecto 1.4

Se pretende mostrar un conjunto de números deslizándose por la matriz de LEDS.

## Hardware necesario

- Micro:bit
- Micro Usb

## Esquema de conexión



## Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

include!("lib_cap.rs");

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut display = Display::new(board.display_pins);

    let mut light_heart_small = [[0; 5];5];
    let data = "0123";

    loop {
        for num in data.chars() {
            lib_cap::number_to_display(num.to_digit(10).unwrap() as u8, &mut
light_heart_small);
            display.show(&mut timer, light_heart_small, 1000);
            for _ in 0..=5 {
                display.show(&mut timer, light_heart_small, 500);
                timer.delay_ms(10_u32);
                lib_cap::rotate_column_matrix(&mut light_heart_small);
            }
        }
    }
}
```

## Comando de ejecución

```
$cargo run --example text_scroll
```

# Código fuente de ayuda

```
mod lib_cap{
  pub fn number_to_display(number:u8 ,matriz:&mut [[u8;5];5]){
    match number{
      0 =>
        *matriz = [
          [0, 1, 1, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 1, 1, 1, 0]],
      1 => {
        *matriz = [
          [0, 0, 1, 1, 0],
          [0, 1, 0, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 0, 1, 0]];
      },
      2 => {
        *matriz = [
          [0, 1, 1, 1, 0],
          [0, 0, 0, 1, 0],
          [0, 0, 1, 0, 0],
          [0, 1, 0, 0, 0],
          [0, 1, 1, 1, 0]];
      },
      _ => {
        *matriz = [
          [0, 0, 0, 0, 0],
          [0, 0, 0, 0, 0],
          [1, 1, 1, 1, 1],
          [0, 0, 0, 0, 0],
          [0, 0, 0, 0, 0]];
      }
    }
  }
}

pub fn rotate_column_matrix(matriz:&mut [[u8;5];5]){
  for col in (1..5).rev() {
    matriz[0][col] = matriz[0][col-1];
    matriz[1][col] = matriz[1][col-1];
    matriz[2][col] = matriz[2][col-1];
    matriz[3][col] = matriz[3][col-1];
    matriz[4][col] = matriz[4][col-1];
  }
  {
    matriz[0][0] = 0;
    matriz[1][0] = 0;
    matriz[2][0] = 0;
    matriz[3][0] = 0;
    matriz[4][0] = 0;
  }
}
```

# Capítulo 2 - Botones

Los teclados o botones son herramientas importantes para la interacción nosotros y el ordenador. A menudo utilizamos los teclados para introducir texto, escribir comandos, controlar dispositivos, etc. La micro:bit incorpora dos botones programables, A y B, que permiten controlarla para que realice acciones.

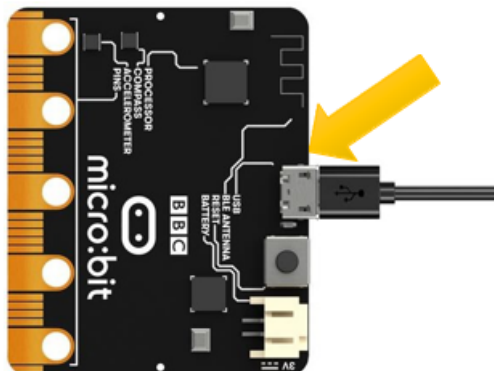
## Descripción del proyecto 2.1

Este proyecto tiene como objetivo mostrar dos patrones diferentes en función del botón que se pulse. Si se pulsa el botón A, se mostrará un patrón de flecha hacia la izquierda en la pantalla LED de la MB2. Si se pulsa el botón B, se mostrará un patrón de flecha hacia la derecha.

### Hardware necesario

- Micro:bit
- Micro Usb

### Esquema de conexión



## Código fuente

```
#![no_main]
#![no_std]

use cortex_m_rt::entry;
// use embedded_hal::delay::DelayNs;
use embedded_hal::digital::InputPin;
use microbit::{board::Board, display::blocking::Display, hal::Timer};
use panic_rtt_target as _;
use rtt_target::rtt_init_print;

#[entry]
fn main() -> ! {
    rtt_init_print!();

    let board = Board::take().unwrap();
    let mut display = Display::new(board.display_pins);
    let mut btn_a = board.buttons.button_a;
    let mut btn_b = board.buttons.button_b;
    let mut timer = Timer::new(board.TIMER0);

    let derecha = [
        [0, 0, 1, 0, 0],
        [0, 0, 0, 1, 0],
        [1, 1, 1, 1, 1],
        [0, 0, 0, 1, 0],
        [0, 0, 1, 0, 0],
    ];
    let izquierda = [
        [0, 0, 1, 0, 0],
        [0, 1, 0, 0, 0],
        [1, 1, 1, 1, 1],
        [0, 1, 0, 0, 0],
        [0, 0, 1, 0, 0],
    ];

    loop {
        if btn_a.is_low().unwrap() {
            display.show(&mut timer, izquierda, 1000);
        } else if btn_b.is_low().unwrap() {
            display.show(&mut timer, derecha, 1000);
        }
    }
}
```

# Capítulo 3 - Led

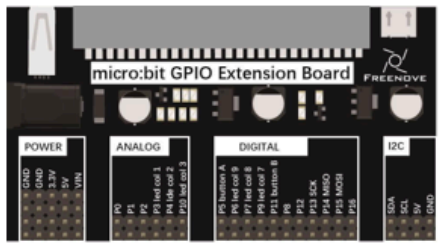
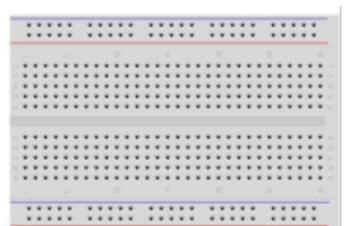
En esta sección aprenderemos a controlar los LEDs de la MB2.

## Descripción del proyecto 3.1

Este proyecto tiene como objetivo de hacer parpadear un único LED externo.

### Hardware necesario

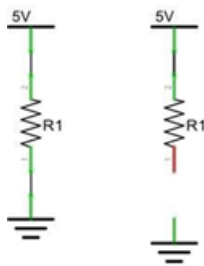
- Micro:bit
- Micro Usb
- Placa de Extensión
- Placa de conexión
- Cable de conexión
- Resistencia de 220 ohmios
- Un Led

|                                                                                     |                                                                                      |                                                                                       |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Micro bit x1                                                                        | Expansion board x1                                                                   |                                                                                       |
|   |   |                                                                                       |
| Break board x1                                                                      | USB cable x1                                                                         |                                                                                       |
|  |  |                                                                                       |
| Jumper F/M x2                                                                       | Resistor 220Ω x1                                                                     | LED x1                                                                                |
|  |   |  |

### Conociendo los componentes

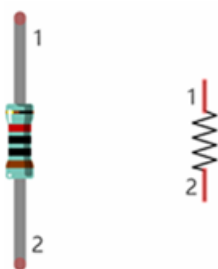
#### Circuito integrado micro:bit

La unidad de intensidad (I) es el amperio (A).  $1\text{ A} = 1\,000\text{ mA}$ ,  $1\text{ mA} = 1\,000\text{ }\mu\text{A}$ . Para que haya intensidad, es necesario un circuito cerrado formado por componentes electrónicos. En la siguiente figura: a la izquierda hay un circuito cerrado, por lo que la intensidad circula por el circuito. A la derecha no hay un circuito cerrado, por lo que no hay intensidad.



## Resistencias

Las resistencias utilizan el ohmio ( $\Omega$ ) como unidad de medida de su resistencia (R).  $1 \text{ M}\Omega = 1\,000 \text{ k}\Omega$ ,  $1 \text{ k}\Omega = 1\,000 \Omega$ . Una resistencia es un componente eléctrico pasivo que limita o regula el flujo de corriente en un circuito electrónico. A la izquierda vemos una representación física de una resistencia, y a la derecha está el símbolo que se utiliza para indicar la presencia de una resistencia en un diagrama de circuito o esquema.



Las bandas de color de una resistencia constituyen un código abreviado que se utiliza para identificar su valor de resistencia. Para obtener más detalles sobre los códigos de colores de las resistencias, consulta la ficha incluida en el paquete del kit.

Con una tensión fija, cuanto mayor sea la resistencia añadida al circuito, menor será la corriente de salida. La relación entre corriente, tensión y resistencia se puede expresar mediante esta fórmula:  $I = V/R$ , conocida como la ley de Ohm, donde  $I$  = corriente,  $V$  = tensión y  $R$  = resistencia. Si se conocen los valores de dos de estos tres parámetros, se puede calcular el valor del tercero. En el siguiente diagrama, la corriente que atraviesa  $R1$  es:  $I = V/R = 5 \text{ V} / 10 \text{ k}\Omega = 0,0005 \text{ A} = 0,5 \text{ mA}$ .



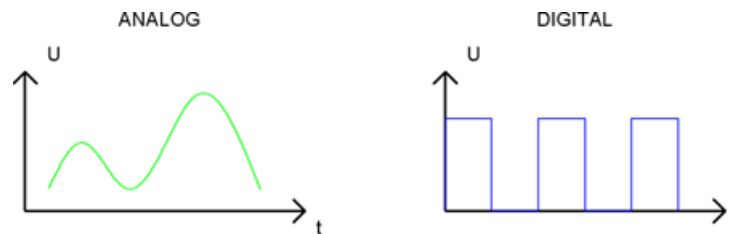
**ADVERTENCIA:** Nunca conectes los dos polos de una fuente de alimentación con ningún elemento de baja resistencia (por ejemplo, un objeto metálico o un cable pelado), ya que esto provoca un cortocircuito y genera una corriente elevada que puede dañar la fuente de alimentación y los componentes electrónicos.

**Nota:** A diferencia de los LEDs y los diodos, las resistencias no tienen polos y son no polares (no importa en qué dirección se inserten en un circuito, funcionarán igual).

## Señal Analógica y Digital

Una señal analógica es una señal continua tanto en el tiempo como en su valor. Por el contrario, una señal digital o de tiempo discreto es una serie temporal formada por una secuencia de magnitudes.

La mayoría de las señales que encontramos en la vida cotidiana son señales analógicas. Un ejemplo conocido de señal analógica sería cómo la temperatura varía continuamente a lo largo del día y no podría cambiar de forma repentina e instantánea de  $0^{\circ}\text{C}$  a  $10^{\circ}\text{C}$ . Sin embargo, el valor de las señales digitales puede cambiar instantáneamente. Este cambio se expresa numéricamente mediante los números 1 y 0 (la base del código binario). Sus diferencias se aprecian más fácilmente al compararlas en un gráfico como el que se muestra a continuación.

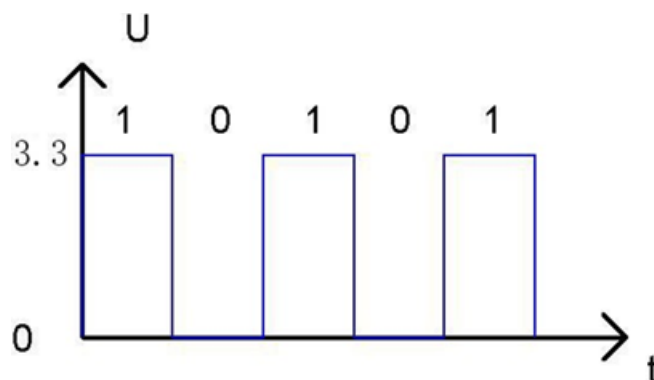


Ten en cuenta que las señales analógicas son ondas curvas y las señales digitales son “ondas cuadradas”. En aplicaciones prácticas, solemos utilizar el sistema binario como señal digital, es decir, una secuencia de ceros y unos. Dado que una señal binaria solo tiene dos valores (0 o 1), ofrece una gran estabilidad y fiabilidad. Por último, tanto las señales analógicas como las digitales pueden convertirse unas en otras.

### Nivel bajo y alto de tensión

En los circuitos, los valores binarios (0 y 1) se representan como nivel bajo y nivel alto. El nivel bajo suele ser igual a la tensión de tierra ( $0\text{ V}$ ). El nivel alto suele ser igual a la tensión de funcionamiento de los componentes.

El nivel bajo del Micro:bit es de  $0\text{ V}$  y el nivel alto es de  $3,3\text{ V}$ , tal y como se muestra a continuación. Cuando el puerto de E/S del Micro:bit emite un nivel alto, se pueden controlar directamente componentes de bajo consumo, como los LEDs.



### Conectores

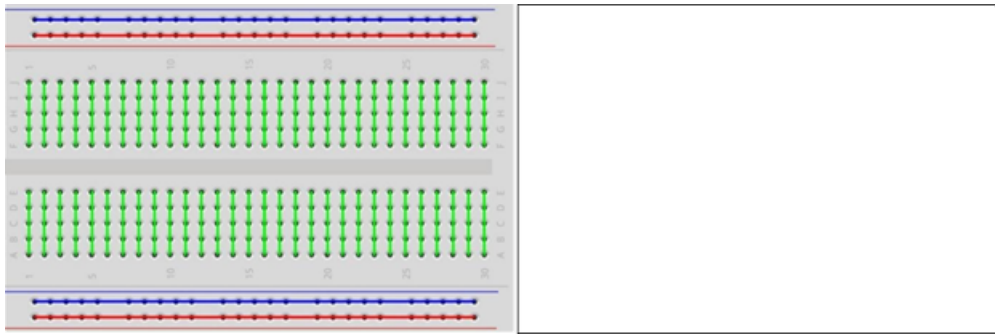
Un “cable conector” es un tipo de cable diseñado para conectar componentes entre sí mediante la inserción de sus dos terminales. Los conectores tienen un extremo macho (pin) y un extremo hembra (ranura), por lo que se pueden clasificar en los tres tipos siguientes.





## Placa de pruebas

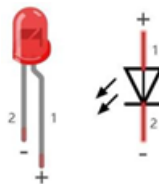
En la placa de pruebas hay muchos orificios pequeños para unir los conectores. Algunos de estos orificios están enlazados entre sí en el interior de la placa. Aquí tenemos una pequeña placa de pruebas a modo de ejemplo de cómo están conectadas eléctricamente las filas de orificios (zócalos). La imagen de la izquierda muestra cómo los pines comparten la conexión eléctrica, y la de la derecha muestra el metal interno que une eléctricamente estas filas.



## Led (Externo)

Un LED es un tipo de diodo. Todos los diodos solo funcionan si la corriente circula en la dirección correcta y tienen dos polos. Un LED solo se iluminará si la patilla más larga (+) del LED se conecta al polo positivo de una fuente de alimentación y la patilla más corta se conecta al polo negativo (-) de la misma, también denominado tierra (GND). Este tipo de componente se conoce como “polar” (piensa en una calle de sentido único).

Todos los diodos comunes de dos terminales son iguales en este aspecto. Los diodos solo funcionan si la tensión de su electrodo positivo es superior a la de su electrodo negativo, y la mayoría de los diodos tienen un rango de tensión de funcionamiento muy reducido, comprendido entre 1,9 y 3,4 V. Si se aplica una tensión muy superior a 3,3 V, el LED se dañará y se fundirá.



---

**Nota:** Los LEDs no se pueden conectar directamente a una fuente de alimentación, ya que esto suele provocar daños en el componente. Es necesario conectar en serie una resistencia con un valor de resistencia específico al LED que se vaya a utilizar.

---

## Esquema de conexión

Al realizar el cableado, se recomienda desconectar todas las fuentes de alimentación del circuito y, a continuación, montar el circuito siguiendo el esquema (la placa micro:bit no se puede insertar al revés), el polo positivo del LED (pin largo) debe conectarse a la resistencia, mientras que su polo negativo (pin corto) debe conectarse a GND. Una vez montado y comprobado que el circuito es correcto, utiliza el cable USB para conectar el ordenador al micro:bit y alimentar el circuito.

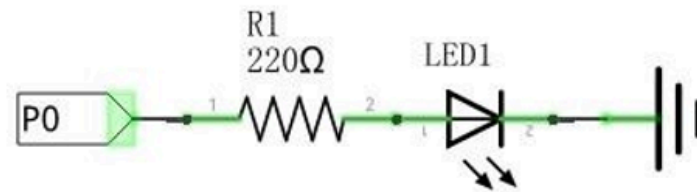
---

**PRECAUCIÓN:** ¡Evita cualquier posible cortocircuito (especialmente al conectar 5 V o GND, 3,3 V y GND)! **ADVERTENCIA:** ¡Un cortocircuito puede provocar una corriente elevada en tu circuito, generar un calor excesivo en los componentes y causar daños permanentes en tu micro:bit!

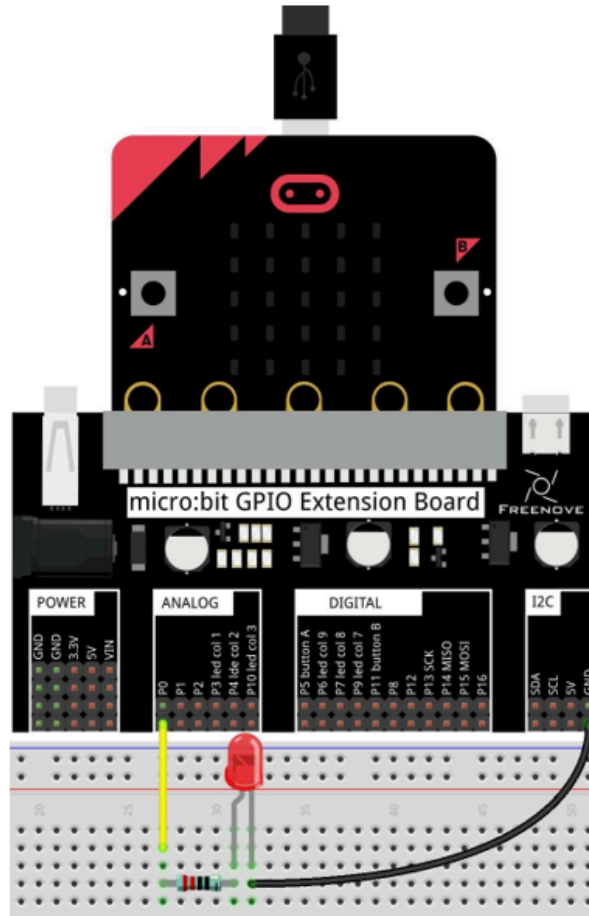
---

**Nota:** Patilla larga del diodo conectada en la misma columna que la resistencia, patilla corta del diodo conectada en la misma columna que GND.

Schematic diagram

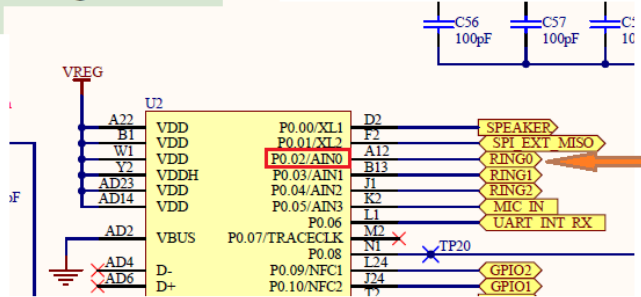


Hardware connection

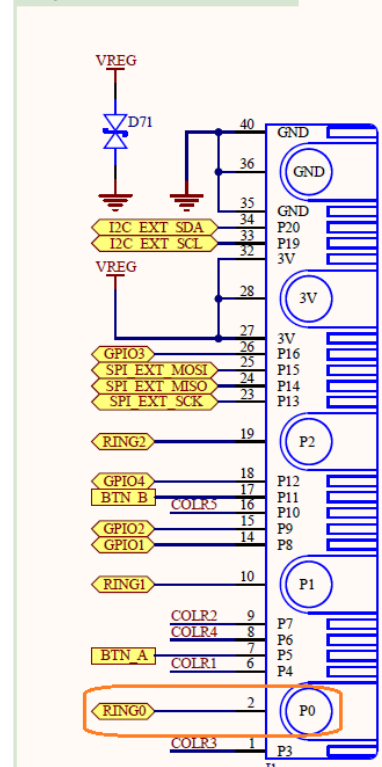


El puerto a usar sería el P0, el pin a utilizar el 02.

## Target MCU



## Expansion connector



## Código fuente

```

#![no_main]
#![no_std]

use cortex_m_rt::entry;
use embedded_hal::delay::DelayNs;
use embedded_hal::digital::OutputPin;
use microbit::Board;
use nrf52833_hal::Timer;
use nrf52833_hal::gpio::Level;
use panic_rtt_target as _;
use rtt_target::{rprintln, rtt_init_print};

#[entry]
fn main() -> ! {
    rtt_init_print!();
    rprintln!("Iniciando programa");
    let board = Board::take().unwrap();
    let mut timer = Timer::new(board.TIMER0);
    let mut led = board.edge.e00.into_push_pull_output(Level::Low);

    loop {
        timer.delay_ms(1000_u32);
        rprintln!("Encendiendo");
        led.set_high().unwrap();
        timer.delay_ms(1000_u32);
        rprintln!("Apagando");
        led.set_low().unwrap();
    }
}

```

# Estructura de las librerías Rust

## Estructura de las librerías Rust

En esta sección aprenderemos a controlar los LEDs de la MB2.

**HAL**

**PAC**

**bcp**